

**AI +
DevOps**



AiDevopsMonitoringAgent

AI-Powered Real-Time Infrastructure Monitoring and Incident Analysis

Ashutosh Thakur | February 2026

The Monitoring Challenge

Modern infrastructure demands real-time visibility, but traditional monitoring approaches create bottlenecks that impact reliability and increase operational overhead.

Manual Monitoring Bottlenecks

Engineering teams spend hours parsing dashboards and logs across distributed systems, leading to delayed detection of critical issues.

Alert Fatigue Crisis

Silos of monitoring tools generate thousands of alerts daily. Teams become desensitized, causing genuine incidents to be overlooked.

Expertise-Dependent RCA

Root cause analysis requires deep domain knowledge spanning multiple system layers, creating dependency on senior engineers.

Disconnected Tooling

Traditional monitoring lacks intelligent correlation between metrics, logs, and traces—forcing engineers to manually connect the dots.

Cost of Downtime

Every minute of service disruption impacts revenue, user trust, and operational stability—making rapid response non-negotiable.

An Intelligent, Automated Solution

AiDevopsMonitoringAgent bridges cutting-edge AI capabilities with DevOps monitoring principles, transforming reactive alerting into proactive observability.



AI-Driven Anomaly Detection

Continuous metric ingestion from Prometheus targets combined with intelligent pattern recognition identifies subtle anomalies before they escalate.



LLM-Powered Root Cause Analysis

Generates human-readable incident summaries and causal chains using Gemini's contextual understanding of your infrastructure.



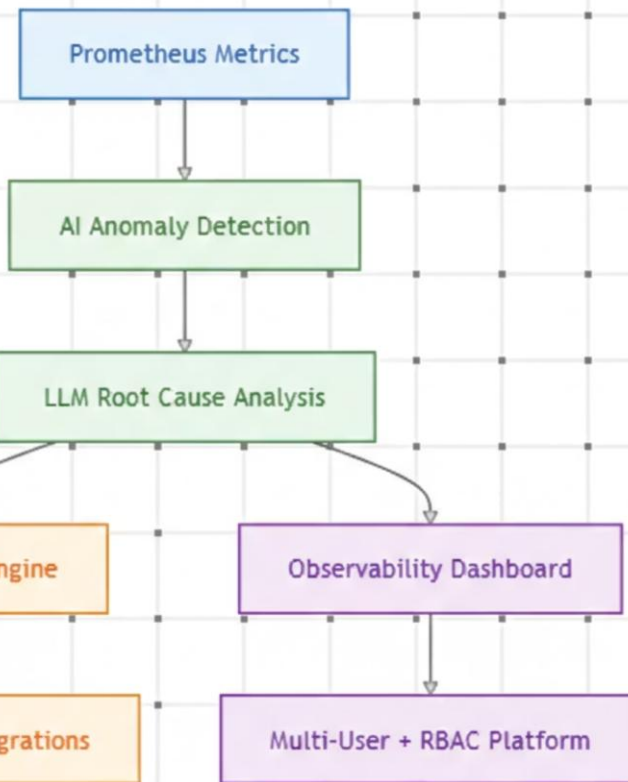
Self-Service Platform

Multi-user architecture with role-based isolation enables teams to configure monitoring independently while maintaining data boundaries.



Intelligent Alert Routing

Automatically dispatches contextual alerts via Email and Slack integrations when thresholds breach, with severity-based escalation paths.



System Architecture Overview

A distributed, cloud-native architecture integrates battle-tested monitoring tools with modern AI/ML pipelines while maintaining operational observability.

01

React Frontend Interface

User dashboard built with React communicates with backend via REST APIs, providing real-time updates and configuration management.

03

Prometheus Scraping Layer

Dynamic target discovery fetches metrics at configurable intervals, supporting file-based service discovery for containerized workloads.

05

MongoDB Data Storage

Structured document storage segregates incidents, anomalies, and RCA data by user for compliance and isolation requirements.

02

FastAPI Backend Engine

Python-based API layer handles authentication, request routing, and orchestrates the monitoring pipeline with async processing.

04

Gemini LLM Analysis

Batched metrics fed to Gemini 2.5 Pro for anomaly classification, severity scoring, and automated root cause generation.

06

Langfuse Observability

Tracks LLM token usage, response latency, and costs—ensuring transparent AI operations and budget optimization.

Technology Stack

Enterprise-grade components selected for performance, maintainability, and production readiness in high-availability environments.

Backend Infrastructure

- **Python 3.11** with type annotations for robust error handling
- **FastAPI** framework providing OpenAPI documentation and validation
- **Uvicorn** ASGI server for high-concurrency request handling
- **Prometheus Client** libraries for metric instrumentation
- **Pydantic** for Data validation and settings management

AI/ML Engine

- **Gemini 2.5 Pro** for primary analysis with 1M+ token context
- **Gemma 3** as fallback model for cost-sensitive queries
- **Langfuse** for LLM tracing and cost attribution

Frontend Stack

- **React 18** with hooks for component state management
- **Vite** as modern build tool with hot module replacement
- **Tailwind CSS** for utility-first responsive styling
- **TypeScript** providing compile-time type safety
- **Recharts** for interactive time-series visualizations

Data & Monitoring

- **MongoDB Atlas** for managed document storage
- **Prometheus** for time-series metric collection
- **Custom Exporters** for proprietary system metrics
- **JSON logging** for structured analysis pipelines

Core Features and Workflow

An automated pipeline that transforms raw metrics into actionable intelligence—reducing incident response time from hours to minutes.



Live Dashboard

 **AI DevOps**
Monitoring

MONITORING

Dashboard

Metrics

Anomalies

RCA Results

SETTINGS

Alerts & Servers

Email Config

Ashutosh

kafjisd@gmail.com

Sign Out

AiDevopsMonitoringDashboard

View Live Metrics

Online

Metric Batches

179

Total Anomalies

177

Open Issues

0

RCA Complete

179

Notifications

Email AlertsInactive

Slack AlertsInactive

Components

API ServerHealthy

MongoDBHealthy

PrometheusHealthy

LLM ServiceHealthy

Adding new server endpoint

Server Management

Configure monitored servers and alert integrations.

Add New Server

Server Name (e.g. Prod DB)

Endpoint (e.g. 192.168.1.5:9100)

Add Server

Monitored Servers

1 active

2nd

ACTIVE

host.docker.internal:8001

Delete

Slack Integration

Enable Slack Alerts

Webhook URL

https://hooks.slack.com/services/...

Save Configuration

Monitoring

MEDIUM node_cpu_usage_percentage

Instance: host.docker.internal:8001

Elevated CPU Usage

Observed

55.75

Expected

< 30%

Detected

07:44:25 pm

17 Feb 2026

Grafana

ID: 69943171... Batch: 69943171... Incident: 69943171...

MEDIUM node_cpu_usage_percentage

Instance: host.docker.internal:8001

High CPU Usage

Observed

55.75

Expected

< 50%

Detected

07:44:23 pm

17 Feb 2026

Grafana

ID: 6994316f... Batch: 6994316f... Incident: 6994316f...

RCA Results

Root cause analysis and recommended actions

Refresh

Total reports

189

RCA Batch Analysis

Instance: host.docker.internal:8001

Ask AI

The node `host.docker.internal:8001` is exhibiting moderately high CPU utilization at 55.75%. While other vital signs like memory, disk, and availability appear normal, this sustained CPU load could indicate an inefficient process, an increased workload, or a precursor to performance degradation. No other correlated anomalies were detected.

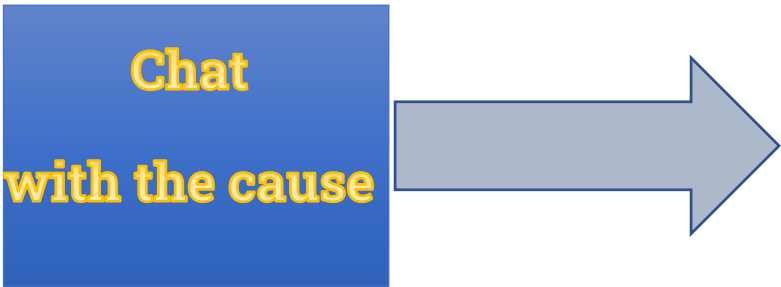
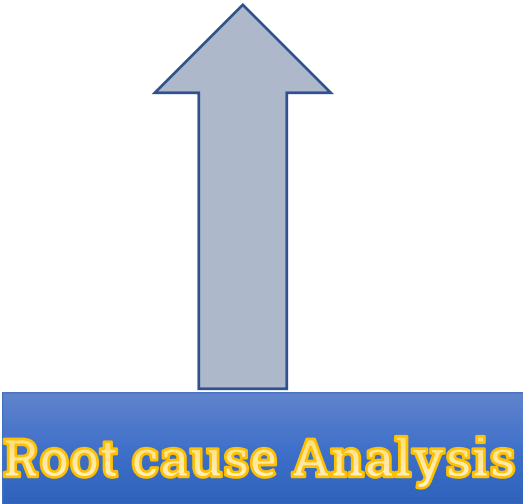
Root cause

The specific root cause is undetermined from the provided metrics alone. The primary suspect is an increased workload or an inefficient process running on the node, as other system resources (memory, disk) and the exporter's availability are stable.

Recommended action

Correlate CPU usage with application-level metrics (e.g., request rate, error rate) to check for user-facing impact., Connect to the host and use tools like `top` or `htop` to identify the specific process(es) consuming high CPU.

RCA: 69943171... 17 Feb 2026, 07:44:25 pm



AI Assistant

Discussing RCA

x

Hi! I analyzed the anomaly RCA. What do you want to understand?

whats happening

Of course. Here's a summary of what's happening:

The node `host.docker.internal:8001` is experiencing moderately high CPU usage (around 56%).

- * **Impact:** While not critical, this could be an early sign of performance issues. Other system resources like memory and disk are stable.
- * **Possible Cause:** The exact cause isn't known, but it's likely an increased workload or an inefficient process running on the machine.
- * **Next Steps:**
 1. Check if this is affecting users by looking at application-level metrics (like request and error rates).
 2. Log into the host and use a tool like `top` or `htop` to identify the specific process consuming the CPU.

Ask about the RCA...

Send

Email Alerts

Configure email notifications for anomaly alerts

Email Alerts

Email notifications are currently disabled

Add Recipient

email@example.com

+ Add

Recipients (1)

ashutosh@revclerx.com

Remove

Send Test Email

Add new recipients in the Email

Get the Alerts

[MEDIUM] High CPU Utilization on host.docker.internal:8001

T

thakurashutosh042003@gmail.com

[MEDIUM] High CPU Utilization on host.docker.internal:8001

To ashutosh@revclerx.com

[MEDIUM] High CPU Utilization on host.docker.internal:8001

Window: 2026-02-17 19:52 -> 19:54 IST

Summary: The instance host.docker.internal:8001 is experiencing sustained high CPU usage at 79.07%, indicating a potential performance bottleneck or an issue with a running application. This can lead to increased latency and service degradation if not addressed.

Root Cause: An unidentified process or application on the instance is consuming excessive CPU resources.

Blast Radius: Services and applications running on the instance host.docker.internal:8001. Potential for user-facing latency if this instance serves production traffic.

Immediate Actions:

- SSH into the instance `host.docker.internal:8001`.
- Use tools like `top`, `htop`, or `ps -eo pcpu,pid,user,args | sort -k 1 -r | head -10` to identify the specific process(es) with high CPU consumption.
- Analyze the logs of the identified application for errors or unusual activity.

Anomalies: 1 | Confidence: 90%

AI and DevOps Integration

Leverages Google's Gemini 2.5 Pro for production-grade AI analysis, combining speed, accuracy, and cost-efficiency for real-world monitoring workloads.



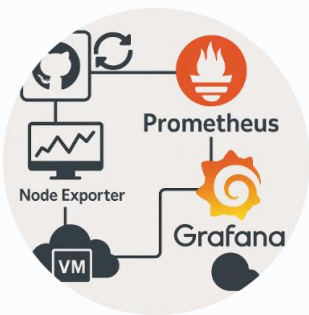
LLM Selection Strategy

Gemini 2.5 Pro provides optimal balance: 128K context window handles complex queries, multimodal support future-proofed, and predictable pricing per 1M tokens.



Natural Language Output

Generates incident descriptions in plain English, translating technical metrics into business-impact statements accessible to all stakeholders—not just SREs.



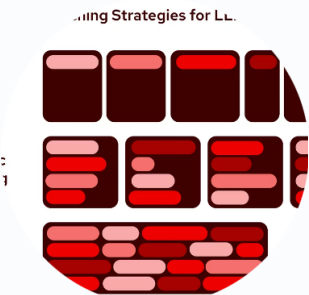
PromQL Integration

Backend translates user questions into PromQL queries.



Dynamic Service Discovery

File-based SD automatically identifies scrape targets as infrastructure changes. JSON configuration reloads without restart—enabling GitOps workflows.



Configurable Batching

Adjust metric collection intervals from 60 seconds to hours based on workload volatility. Longer intervals reduce costs; shorter windows enable faster detection.

Security, Scalability, and Reliability

Enterprise-grade security controls and horizontal scaling architecture ensure the system remains available and secure as infrastructure grows.

Authentication & Access Control

- **JWT Authentication** with refresh token rotation every 7 days prevents session hijacking
- **Argon2 Password Hashing** provides memory-hard security against GPU cracking
- **API Rate Limiting** using sliding window counters to prevent abuse

Data Isolation & Tenancy

- **Database-Level Enforcement** adds WHERE clauses automatically—preventing accidental leaks
- **Encrypted at Rest** using MongoDB Atlas transparent data encryption with customer-managed keys



Stateless Backend

FastAPI instances process requests independently, allowing horizontal scaling via Kubernetes replicas as traffic increases.



Auto-Scaling Storage

MongoDB Atlas automatically provisions additional shards based on metric volume—no manual intervention needed.



Async Processing

Non-blocking metric analysis ensures frontend stays responsive even during anomaly storms.

Challenges and Future Enhancements

Continuous innovation roadmap addresses current technical constraints while expanding platform capabilities for next-generation infrastructure.

Current Limitations

LLM Latency Impact

Gemini API response times (200-500ms) add delay to real-time detection. Mitigated via batch processing but may miss sub-second anomalies.

Token Constraints

Context window limits metrics volume per analysis batch. Large clusters require strategic metric selection to stay within budget.

Planned Features

Predictive Alerting

Historical trend analysis using time-series forecasting to predict threshold breaches before they occur—shifting from reactive to proactive.

Metric Filtering

Enable advanced metric filtering by service, environment, severity, or time range to reduce noise and improve analysis precision.